Tecnológico de Monterrey
Escuela de Ingeniería y Ciencias

Campus Santa Fe

Construcción de software y toma de decisiones

TC2005B.401

“Evidencia.1. Evidencia entrega proyecto final”

Grupo: 401

Integrantes:

Daniela Angulo Solís	A01028153
Sofia Campos	A01277013
Ariel Pulver	A01786822

Fecha de Entrega:

30 de abril del 2026

¿Por qué nuestro juego?

Al principio sabíamos que queríamos hacer algo de combate y progresión, pero necesitábamos una estructura que no nos obligara a crear contenido infinito ni una historia compleja. Por eso terminamos eligiendo un roguelite side-scroller en Roma: el formato de runs nos resolvía problemas de diseño, y la ambientación romana ya trae contexto por sí sola (gladiadores, coliseo, público, emperador)**.**

Lo interesante del juego fue apareciendo conforme lo fuimos construyendo, con la implementación de nuestros 2 sistemas principales:

- El **sistema de fama y target time**l: no solo se trata de sobrevivir o avanzar, sino de gestionar el riesgo. Obtener más fama puede darte ventajas (como mejoras o recursos), lo que obliga al jugador a tomar decisiones constantemente sobre si debe arriesgarse a ejecutar el nivel con la mayor rapidez posible o seguir por el camino seguro, aunque las recompensas no son tan grandes.

- El **sistema de cartas (power-ups y punishments)** hace que cada partida sea diferente. Aunque el jugador siempre recibe opciones, nunca tiene control total, lo que genera variabilidad y rejugabilidad. Además, el hecho de elegir entre beneficios y consecuencias refuerza el tema central de riesgo vs recompensa dentro del contexto del juego.

Y los 3 arquetipos ayudaron a que desde el inicio el jugador sintiera que puede jugar de formas distintas.

Esta estrategia nos dio una estructura clara entre escenas separadas (menú, selección, niveles, resultados), lo cual facilita la implementación técnica. También nos permitió integrar backend con frontend de forma significativa (por ejemplo: guardar partidas, estadísticas, progreso del jugador, tienda, etc.), conectando con bases de datos reales, e hizo posible implementar múltiples mecánicas interactivas (combate, progresión, decisiones, economía dentro del juego).

Retos y Soluciones

Con Git, tuvimos conflictos al trabajar en paralelo en tres ramas distintas, una por integrante. El problema principal era que varios archivos clave como **levelBase.js**, **server.js** y **shopScene.js** eran modificados por diferentes personas al mismo tiempo. Lo resolvimos organizando mejor las ramas, definiendo quién trabajaba en qué parte del código, y resolviendo los conflictos manualmente archivo por archivo al hacer merge.

En la base de datos, hubo varios errores. Algunos datos no se guardaban correctamente porque la tabla Player no tenía las columnas fame, hearts y galen desde el principio. Se fueron agregando conforme el juego lo necesitaba con ALTER TABLE. También tuvimos problemas con queries que fallaban silenciosamente porque referenciaban columnas que no existían, como p.galen antes de agregarla. Lo solucionamos ajustando los endpoints del servidor, validando las respuestas del backend y sincronizando mejor los datos entre window.loggedPlayer y la base de datos.

En Canvas, tuvimos bugs de renderizado donde los sprites no cargaban o se deformaban. Esto pasaba porque las rutas de los assets estaban mal escritas o porque se intentaba dibujar antes de que las imágenes terminaran de cargar. Se corrigió revisando las rutas de los assets y asegurando que los sprites se inicializaran correctamente en el constructor de cada clase.

En el sistema de cartas, la lógica no siempre aplicaba bien los efectos porque los operadores matemáticos y las propiedades que modificaban venían directamente de la base de datos. Lo resolvimos separando la lógica de aplicación y reversión de efectos en funciones independientes (applyEffect y reverseEffect) y asegurándonos de que cada carta tuviera definidos su operador, su valor y su objetivo (jugador, enemigo o juego). En general, fuimos iterando y debuggeando hasta estabilizar cada sistema.

Lecciones Aprendidas

Durante este proyecto aprendimos varias cosas que no sabíamos antes o que no habíamos aplicado en la práctica. Aprendimos a trabajar con Git en equipo de verdad, no solo hacer commits individuales, sino manejar ramas, resolver conflictos de merge y coordinar quién trabaja en qué parte del código para evitar pisarnos el trabajo.

Aprendimos a conectar un frontend con un backend real. Antes podíamos hacer páginas web estáticas, pero en este proyecto tuvimos que hacer fetches, manejar respuestas del servidor, sincronizar el estado del juego con la base de datos y manejar errores de red.

Aprendimos a diseñar una base de datos que realmente se usa. No solo crear tablas, sino pensar en las relaciones entre ellas, usar vistas para simplificar queries complejas, triggers para automatizar actualizaciones y constraints para proteger la integridad de los datos.

Aprendimos que el Canvas de HTML5 es muy diferente a trabajar con el DOM. Todo se dibuja manualmente frame por frame, lo que requiere pensar diferente, manejar el game loop, el deltaTime, las animaciones por spritesheet y la detección de colisiones desde cero.

Y por último, también aprendimos que separar bien la lógica en funciones y archivos independientes hace que el código sea mucho más fácil de mantener y de debuggear cuando algo falla

Conclusiones del equipo

Como conclusiones del equipo, creemos que el proyecto logró lo que buscábamos: no solo entregar algo funcional, sino construir un sistema coherente de principio a fin. A pesar de los cambios que hicimos en la idea original y los problemas que fueron surgiendo, el resultado final se siente consistente, con una lógica clara y una experiencia de juego que sí refleja las mecánicas que planteamos desde el inicio.

También opinamos que una buena organización hace toda la diferencia. Hubo momentos donde el avance se frenaba no por dificultad técnica, sino por falta de coordinación o claridad, y cuando mejoramos eso, todo empezó a fluir mucho más rápido. Esto nos dejó claro que en proyectos así, la comunicación y la estructura del equipo son igual de importantes que el código.

En general, nos quedamos con una experiencia positiva. Fue un bloque exigente, con varios retos, pero también fue de los más completos porque nos obligó a integrar todo en un solo proyecto. Al final, más que solo “cumplir”, sentimos que construimos algo bien hecho y que sí representa el esfuerzo del equipo.